

METHODS

Method is a **block of instructions** to perform a specific task. Method is a class member and always declared in the class block. A method cannot be declared inside another method. The most familiar method is the Main method.

SYNTAX:

The method definition looks like this:

```
[modifiers] return_type identifier ( [formal arguments] )  
{  
}
```

PARTS OF A METHOD:

The method has four parts as follows:

1. Method Signature

identifier ([formal arguments])

2. Method Declaration

[modifiers] identifier ([formal arguments])

3. Method Implementation or Method Block

It is the body of the method which looks like:

```
{  
}
```

4. Method Definition

It is the complete method syntax.

[modifiers] return_type identifier ([formal arguments])

```
{  
}
```

Terminologies in the Method Syntax:

1. Modifiers:

Modifiers are keywords in Java, that are used to change the behaviour of the class members like variables and methods. They are of two types, Access modifiers and non-access modifiers.

Access Modifiers or Access Specifiers:

They are used to set the level of accessibility to the class members. There are three access modifiers and default.

private: can be accessed only within the class.

default: when no access modifier is used. Only within the package.

protected: can be accessed within package and in sub(child) classes of other packages.

public: can be accessed inside and outside class, inside and outside package. Zero restriction.

Non-Access Modifiers:

They are keywords used to change the behaviour of the class members. Important modifiers we are about to use are:

static, abstract, final, synchronized

If no modifier is used it is called non-static member.

2.return type:

Return type of a method is used to specify what type of data will be returned by the method to the method caller when the execution of the method is completed.

Return type is of three types. They are:

1. void – means the method does not return anything
2. Primitive datatype – means returns specified primitive datatype
3. Non-primitive datatype – means returns specified non-primitive type

3.identifier:

It is the method name given by the programmer.

CHARACTERISTICS OF A METHOD:

1. A method gets executed only when it is called.
2. A method can be executed any number of times. Re-usability of code.

TYPES OF METHODS:

There are three types of methods in Java. They are:

1. No argument method
2. Parameterized method
3. Variable argument method – accepts variable arguments Eg: main method

No-argument method:

A method declared with no formal arguments is called No-argument method.

It is easy to call this method, but the drawback is that the user cannot pass data to this method.

SYNTAX:

```
[modifiers] return_type identifier ()  
{  
}
```

A no-argument method looks like this:

```
1  class ClassName  
2  {  
3      public static void abc()  
4      {  
5      }  
6  }  
7  }  
8
```

In the above example, abc() is the no-argument method.

Parameterized method:

A method declared with formal arguments is called Parameterized method. The user can pass data when calling the method, but the drawback is that passing data is mandatory.

SYNTAX:

```
[modifiers] return_type identifier (formal arguments)  
{  
}
```

A parameterized method looks like this:

```
1  class ClassName  
2  {  
3      public static void xyz(int a, int b)  
4      {  
5      }  
6  }  
7  }  
8
```

In the above example, xyz(int, int) is the parameterized method.

CALLING A METHOD:

It is important to call a method, because a method will be executed only when it is called. The method can be called using **method call statement**.

SYNTAX:

The Syntax for calling a method is method signature followed by terminator.

method_name ([actual arguments]);

```
class ClassName
{
    public static void main(String[] args)
    {
        abc(); // METHOD CALL STATEMENT
    }
    public static void abc() // METHOD SIGNATURE
    {
        System.out.println("hello from abc method");
    }
}
```

In the above example, we have called the method abc(), from the main method using the method call statement (**abc();**)

What happens when a method call statement is encountered:

1. The execution of the current method (here, main method) is paused.
2. The control goes to the called method (here, abc() method)
3. The called method is executed.
4. After execution of the called method is completed, the control comes back to the method call statement.
5. The execution of the calling method (here, main method) is resumed.

Method Binding:

It happens when a method is called. The mapping of the method call statement with the method signature is called method binding.

1. If it is a no-argument method in the method call statement, binding happens with no-argument method and it gets executed.
2. If it is a parameterized method, two criteria are checked.
 - Number of formal arguments and number of actual arguments should be same.
 - Datatype of formal arguments and actual arguments should be same and in the same order.

Only then method binding will take place for the parameterized methods.

RETURN TYPE AND RETURN STATEMENT :

The return type is a terminology in the syntax of a method. It is the datatype of the value that is returned by the method. Let us understand with an example.

```
1  class ClassName
2  {
3      public static void main(String [] args)
4      {
5          add(5,3); // METHOD CALL STATEMENT
6      }
7      public static int add(int a, int b) // METHOD SIGNATURE
8      {
9          return a+b;
10     }
11 }
```

In the above example, add(int, int) is a method that can accept two actual arguments of int type and perform addition of two values. After the method is executed, it will return the sum of two numbers back to the caller.

Two changes are important to notice:

- ❖ return type of the method is int
- ❖ return statement is used

The method call statement has two int values as actual arguments in the main method. When execution happens, method binding occurs and the values 5 and 3 are mapped with variables a and b respectively. The return type of the method should be appropriately used by the programmer.

The return statement has return keyword. In this case, it returns the sum.

CHARACTERISTICS OF RETURN STATEMENT:

1. The return statement should be used inside the method block only.
2. Only one return statement can return the value from the method.
3. It has to be used as the last statement of the method.
4. The return statement can return only one value.
5. The return statement cannot be used when the return type of the method is void.
6. The return statement is mandatory if return type of the method is other than void.
7. 'return' is a keyword and the return statement is a control transfer statement.

HOW TO CONSUME THE DATA RETURNED BY A METHOD:

The data returned by the `add(int, int)` method in the above example is not utilized or consumed in the program. There are two ways to consume the value returned:

1. Directly print the data by using the method call in the printing statement
`System.out.println(add(5,3));`
2. Store the data in a variable and either print or keep it for future usage
`int result = add(5,3);`
`System.out.println(result);`

```
1  class ClassName
2  {
3      public static void main(String [] args)
4      {
5          System.out.println(add(5,3)); // print directly
6          int res = add(4,2);
7          System.out.println(res); // store value in a variable and print
8      }
9      public static int add(int a, int b)
10     {
11         return a+b;
12     }
13 }
```

CALLING A METHOD FROM VARIOUS SCOPES:

1. From same class:
 - ❖ method signature can be used
 - ❖ Classname.method signature can be used
2. From different class of same package:
 - ❖ Classname.method signature has to be used
3. From different package:
 - ❖ packagename.Classname.method signature has to be used

```
1 package javapackage;
2
3 class Box{
4     public static void demo() {
5         System.out.println("demo method");
6     }
7 }
8
9 class Sample1 {
10
11     public static void main(String[] args) {
12         abc(); //SAME CLASS
13         Sample1.abc(); //SAME CLASS
14         Box.demo(); //DIFFERENT CLASS AND SAME PACKAGE
15         collectionInterface.Sample.sampledemo(); //DIFFERENT PACKAGE
16     }
17     public static void abc() {
18         System.out.println("abc method");
19     }
20 }
```